

Lab 02: ARP Spoofing Attack

Amar Telidji University — Laghouat | 2025/2026

Student: Zernouh Lakhdar

1. Objectives

The goal of this lab is to understand and demonstrate the ARP Spoofing attack and how it can be used to intercept HTTPS traffic. The lab also explores how to protect against such attacks using a proper Certificate Authority (CA) infrastructure.

The lab uses three virtual machines:

- web.zernouh.local — the legitimate HTTPS server
- hackweb (Kali Linux) — the attacker machine
- linux.ing4.dz — the victim client machine

2. Lab Environment

2.1 Network Configuration

All three VMs are connected on the same Internal Network in VirtualBox with the following IP addresses:

VM Name	Role	IP Address	OS
web.zernouh.local	Legitimate Web Server	10.10.0.100	Ubuntu
hackweb (Kali)	Attacker Machine	10.10.0.2	Kali Linux
linux.ing4.dz (client)	Client	10.10.0.1	Ubuntu

3. Setting Up the Legitimate Server

3.1 Installing Apache with SSL

On the machine web.zernouh.local (10.10.0.100):

```
sudo apt update && sudo apt install apache2 -y
sudo a2enmod ssl
sudo a2ensite default-ssl
sudo systemctl restart apache2
```

3.2 Creating the Legitimate Web Page

A custom index.html page was created so the client can identify the legitimate server:

```
sudo tee /var/www/html/index.html << 'EOF'
<h1>Welcome to the LEGITIMATE server www.zernouh.local</h1>
EOF
```

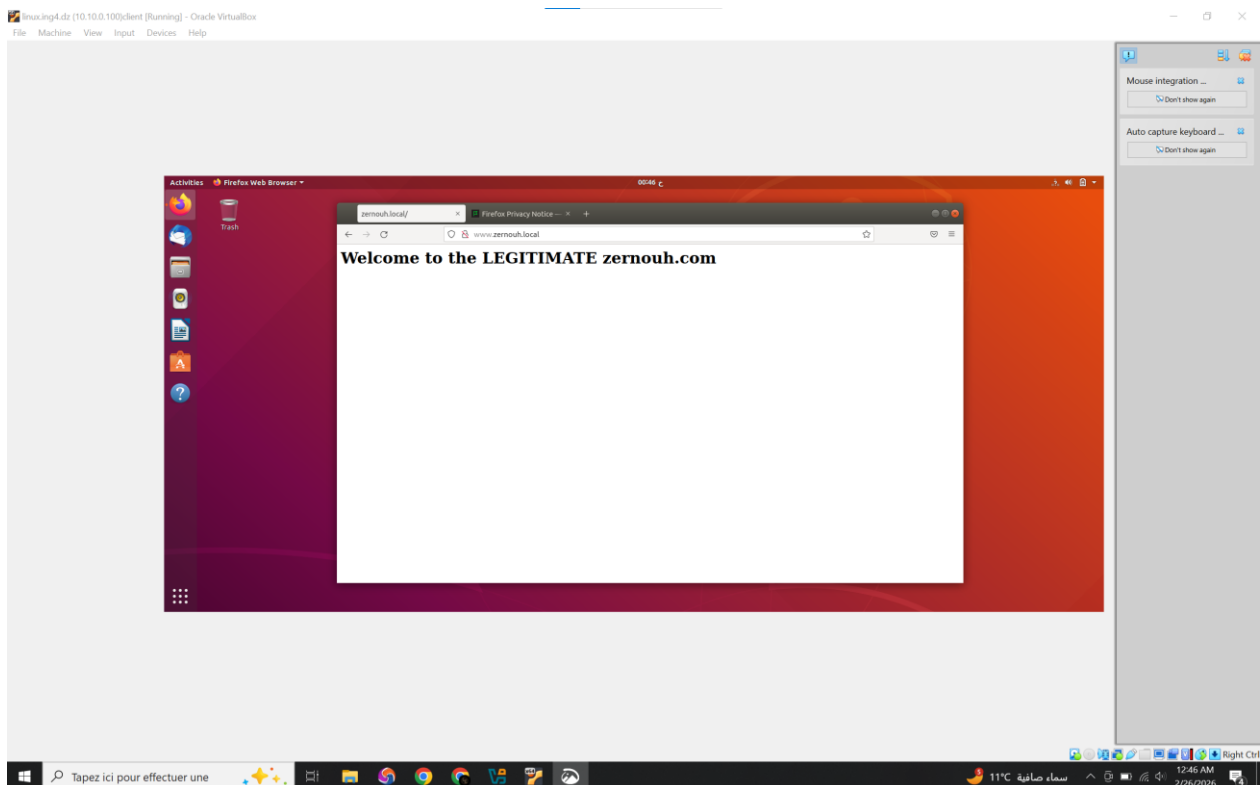
3.3 Hostname Resolution

Since no DNS server was configured, /etc/hosts was used on each machine:

```
# On client and server machines:
sudo nano /etc/hosts
# Add: 10.10.0.100    www.zernouh.local web.zernouh.local
```

3.4 Verifying the Legitimate Server

From the client machine (10.10.0.1):



4. ARP Spoofing Attack

4.1 Concept

ARP (Address Resolution Protocol) maps IP addresses to MAC addresses. Since ARP has no authentication mechanism, an attacker on the same network can send fake ARP replies to poison the ARP table of a victim, redirecting traffic intended for a legitimate host to the attacker's machine.

4.2 Setting Up the Attacker Machine (Kali — 10.10.0.2)

Install required tools:

```
sudo apt install dsniff apache2 -y
sudo a2enmod ssl
sudo a2ensite default-ssl
sudo systemctl restart apache2
```

Create the fake web page:

```
sudo tee /var/www/html/index.html << 'EOF'
<h1>You are on the FAKE ATTACKER server hackweb.zernouh</h1>
EOF
```

4.3 Launching the Attack

Step 1 — Enable IP forwarding:

```
echo 1 | sudo tee /proc/sys/net/ipv4/ip_forward
```

Step 2 — Configure virtual interface with the legitimate server's IP:

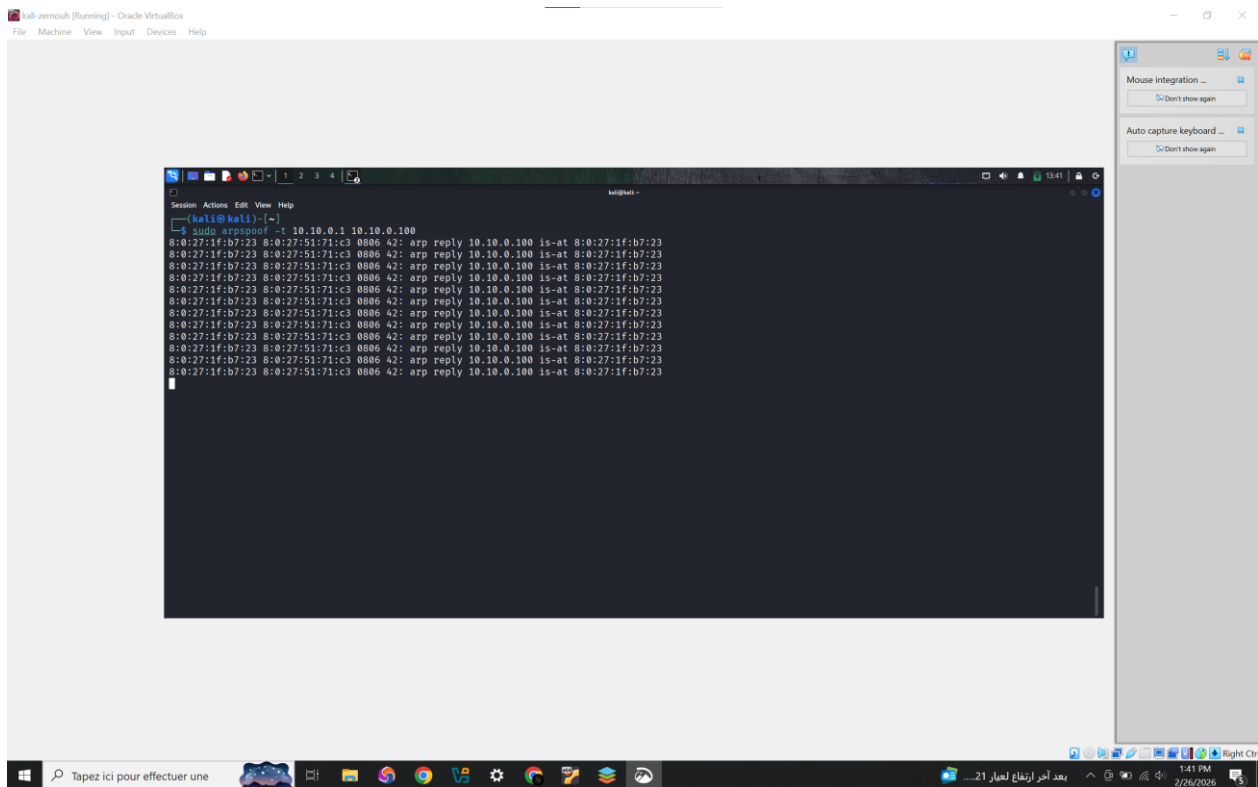
```
sudo ifconfig eth0:0 10.10.0.100
```

This allows the Kali machine to accept packets sent to 10.10.0.100 (the legitimate server's IP).

Step 3 — Launch arpspoof:

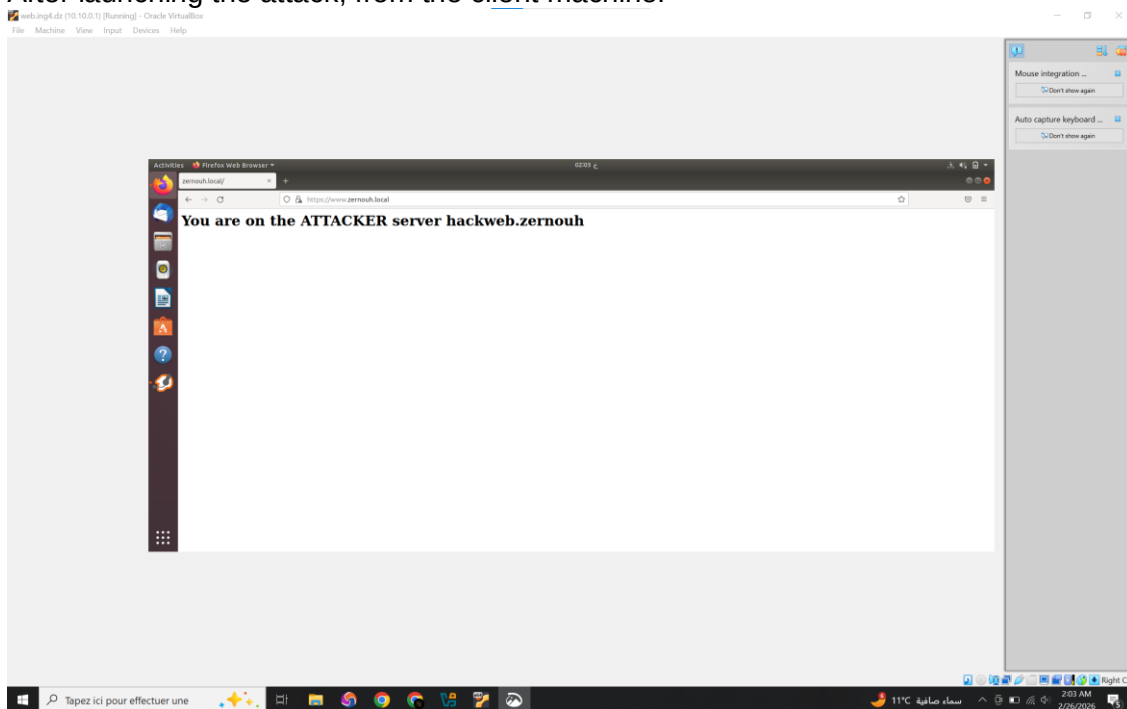
```
sudo arpspoof -t 10.10.0.1 10.10.0.100
```

This sends forged ARP replies to the client (10.10.0.1), telling it that the MAC address associated with 10.10.0.100 (legitimate server) is actually Kali's MAC address.



4.4 Attack Result

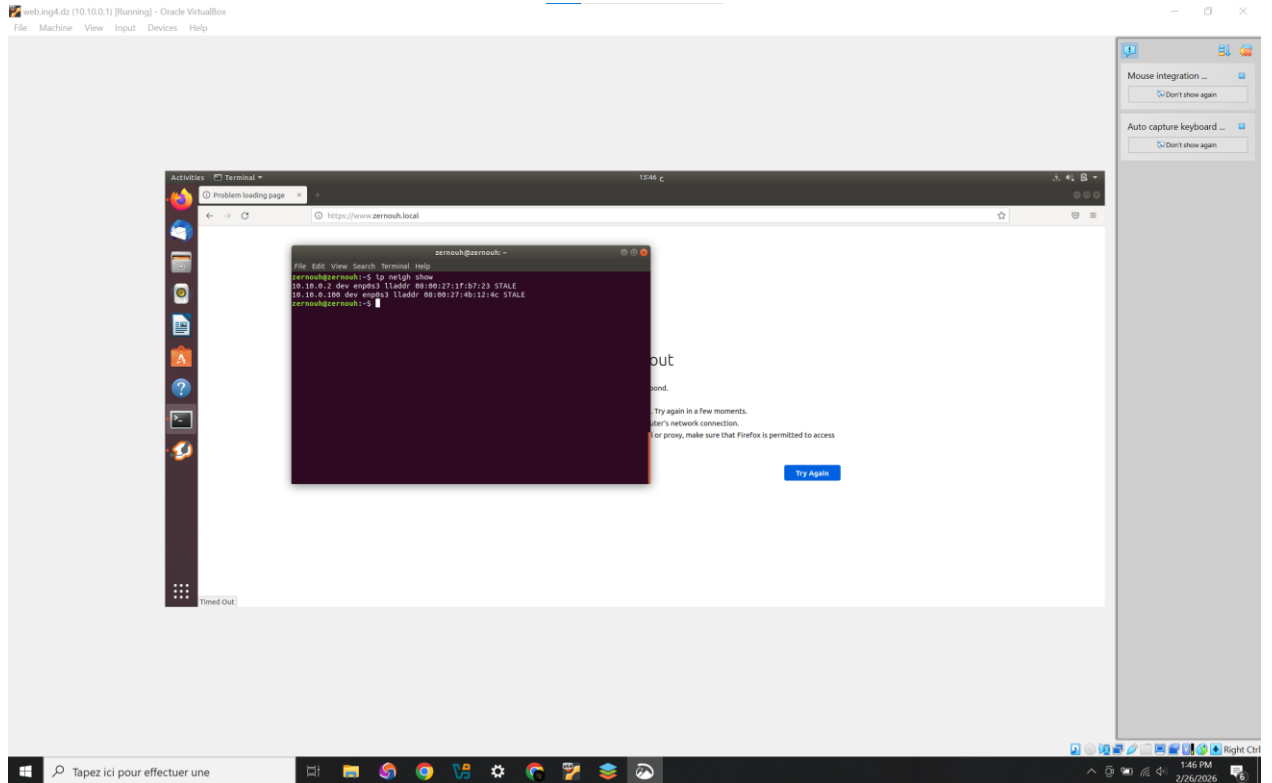
After launching the attack, from the client machine:



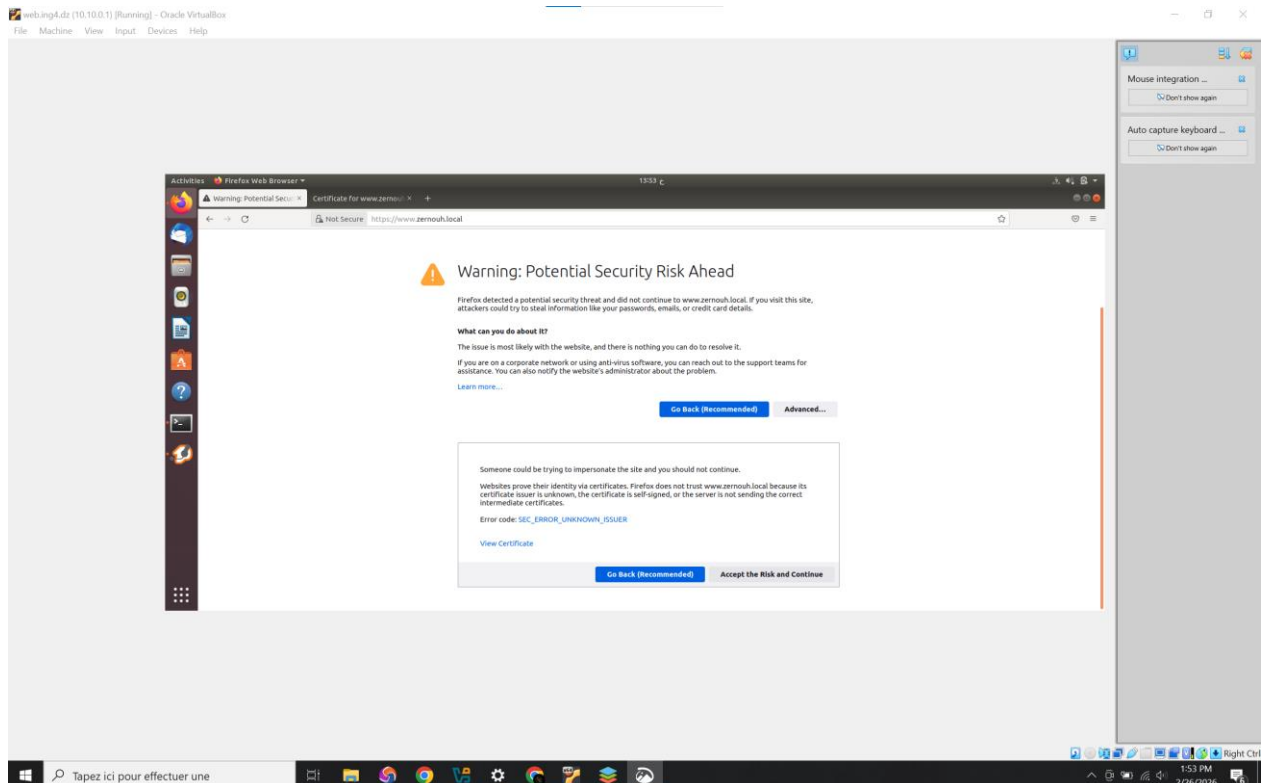
Reuselt : he show the fake web site of the kali

Observations:

- The client's ARP table was poisoned — it now maps 10.10.0.100 to Kali's MAC address



- All HTTPS traffic intended for the legitimate server was redirected to Kali
- HTTPS encryption was still active, but the client was communicating with the attacker
- The browser showed a certificate warning (self-signed cert) — if the user clicks 'Accept', they are fully compromised



Conclusion: HTTPS alone does not protect against ARP spoofing when self-signed certificates are used. The encryption works, but the client has no way to verify they are talking to the correct server.

5. Securing the HTTPS Connection with a Certificate Authority

5.1 Why Self-Signed Certificates Are Insufficient

A self-signed certificate is generated and signed by the server itself — not by a trusted third party. The client cannot verify its authenticity, making it easy for an attacker to present their own self-signed certificate during a spoofing attack.

5.2 Creating a Certificate Authority (CA)

On the legitimate server (10.10.0.100):

Generate the CA private key:

```
openssl genrsa -out myCA.key 2048
```

Generate the CA certificate (self-signed, valid 10 years):

```
openssl req -x509 -new -nodes -key myCA.key -sha256 -days 3650 \
-out myCA.pem \
-subj "/C=DZ/ST=Laghouat/O=Zernouh-CA/CN=Zernouh Certificate Authority"
```

5.3 Creating the Server Certificate Signed by the CA

Generate server key and CSR:

```
openssl genrsa -out server.key 2048
openssl req -new -key server.key -out server.csr \
-subj "/C=DZ/ST=Laghouat/O=Zernouh/CN=www.zernouh.local"
```

Create extension file (required for modern browsers):

```
cat > server.ext << 'EOF'
authorityKeyIdentifier=keyid,issuer
basicConstraints=CA:FALSE
keyUsage = digitalSignature, nonRepudiation, keyEncipherment,
dataEncipherment
subjectAltName = @alt_names
[alt_names]
DNS.1 = www.zernouh.local
EOF
```

Sign the server certificate:

```
openssl x509 -req -in server.csr -CA myCA.pem -CAkey myCA.key \
  -CAcreateserial -out server.crt -days 825 -sha256 -extfile server.ext
```

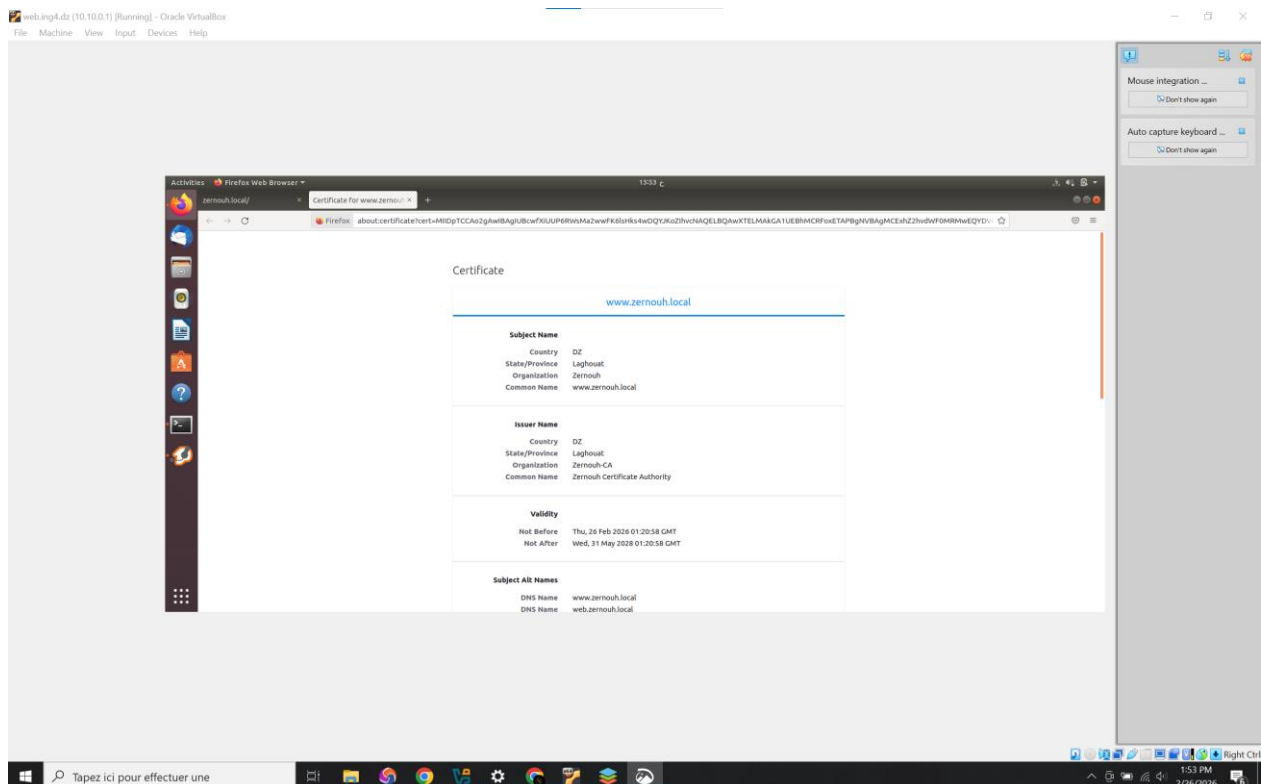
5.4 Configuring Apache to Use the New Certificate

```
sudo cp server.crt /etc/ssl/certs/
sudo cp server.key /etc/ssl/private/
sudo nano /etc/apache2/sites-enabled/default-ssl.conf
```

Update the following lines in the config:

```
SSLCertificateFile /etc/ssl/certs/server.crt
SSLCertificateKeyFile /etc/ssl/private/server.key
sudo systemctl restart apache2
```

This is the CA of zernouh.local



5.5 Making the Client Trust the CA

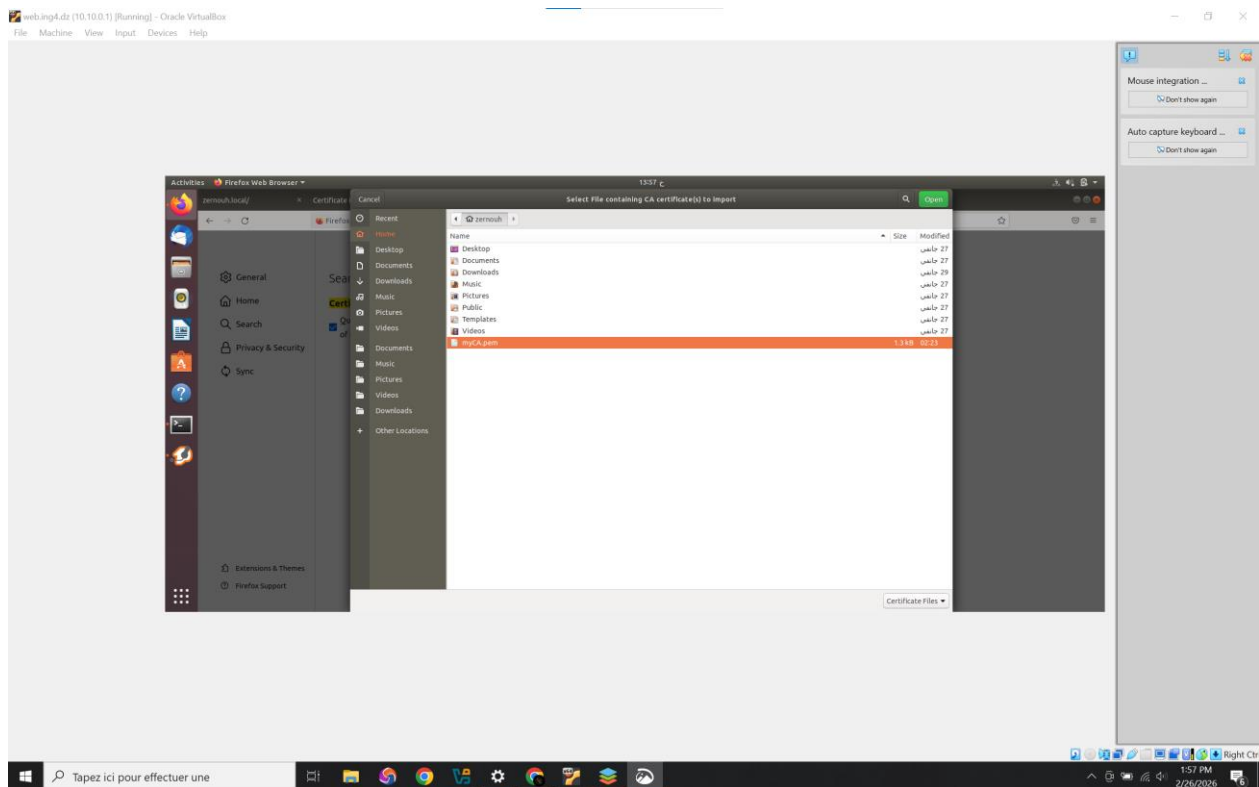
Copy the CA certificate to the client machine:

```
scp myCA.pem user@10.10.0.1:/home/user/
```

On the client machine:

```
sudo cp myCA.pem /usr/local/share/ca-certificates/myCA.crt
sudo update-ca-certificates
```

For Firefox: Settings → Privacy & Security → Certificates → View Certificates → Authorities → Import myCA.pem

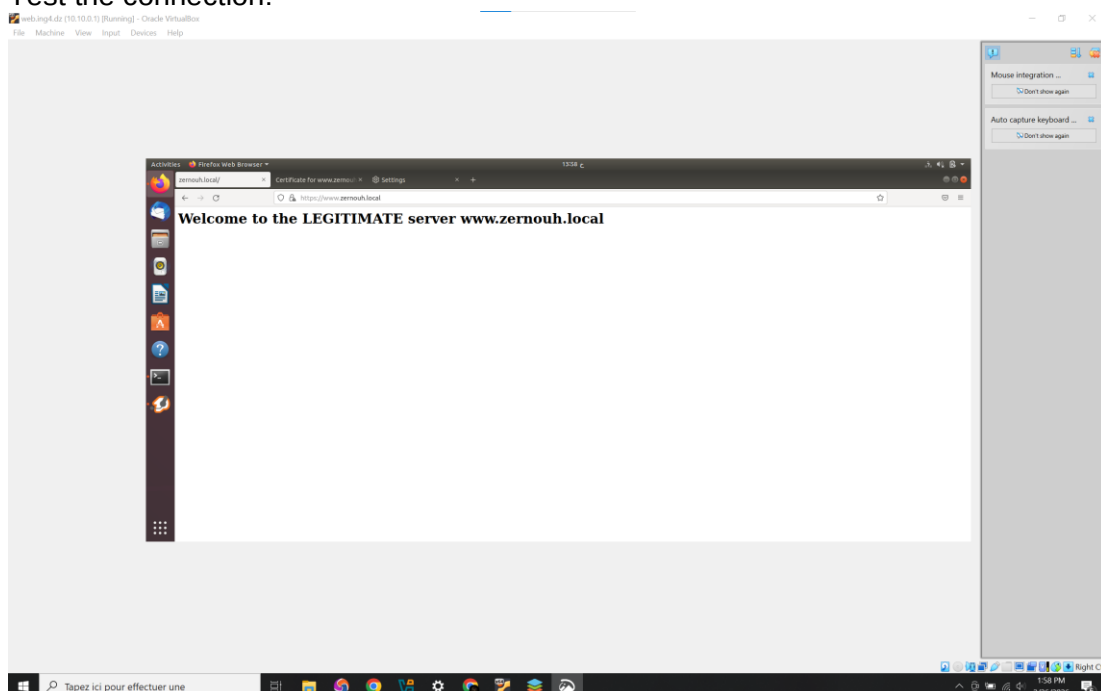


5.6 Testing After CA Setup

Stop the ARP attack on Kali and clear the ARP cache on the client:

```
sudo arp -d 10.10.0.100
```

Test the connection:



Result: Connection succeeded with no certificate warning — the CA-signed certificate was automatically trusted.

If we repeat the ARP spoofing attack:

The attack now fails. The browser/curl rejects the attacker's certificate because it is not signed by the trusted CA.

6. Results Summary

Scenario	Result	Explanation
Self-signed cert, no attack	✓ Connected (with warning)	Browser warns about self-signed cert
Self-signed cert + ARP spoof	✗ Attacker intercepts	Client accepts fake cert if exception clicked
CA-signed cert + ARP spoof	✓ Attack fails	Browser rejects attacker's invalid certificate

7. Conclusion

This lab demonstrated several important security concepts:

- ARP has no built-in authentication, making ARP spoofing trivially easy on a local network
- HTTPS encrypts traffic but does not prevent a man-in-the-middle attack if certificate validation is bypassed
- Self-signed certificates offer no real protection against impersonation attacks
- A properly deployed Certificate Authority solves the key distribution problem and prevents ARP spoofing from compromising HTTPS connections
- Users should never click 'Accept' on unknown certificate warnings — this is exactly what attackers rely on

The Certificate Authority model works because: the client only needs to trust a small set of authorities (pre-installed in browsers), and these authorities ensure that only the legitimate domain owner can obtain a valid certificate for that domain. Even if an attacker redirects traffic, they cannot forge a certificate signed by the trusted CA.